

MULTI-AGENT RESEARCH INTELLIGENCE

Research-Mind InsightForge

From scratch to production deployment

Interview-oriented project handbook in Hinglish

Problem → Evidence collection → Multi-agent analysis → Cited report

OpenAlex + Google News + optional Tavily

Gemini research + optional Groq critic

Streamlit + LangChain + Docker + Streamlit Cloud

Prepared as a complete revision and interview guide

No API keys or private credentials are included in this document.

Contents

1	Sabse Pehle: 60-Second Project Summary	4
1.1	Problem kya solve hoti hai?	4
1.2	English elevator pitch	4
1.3	Hinglish elevator pitch	4
1.4	Project ke strongest differentiators	4
2	Project Kahan Se Start Hua: Actual Evolution	5
2.1	Starting point	5
2.2	Idea pivots aur final requirement	5
2.3	Actual refactoring order	5
3	Requirements Se Architecture Tak	6
3.1	Functional requirements	6
3.2	Non-functional requirements	6
3.3	High-level architecture	7
3.4	Why sequential pipeline?	7
4	Technology Stack: Kya Use Kiya Aur Kyun	7
4.1	Why not LangGraph?	8
5	File-by-File Deep Walkthrough	8
5.1	<code>agents.py</code> : intelligence layer	8
5.1.1	Simple model setup	8
5.1.2	Prompt design principles	9
5.2	<code>tools.py</code> : live evidence layer	9
5.2.1	OpenAlex collector	9
5.2.2	Google News RSS collector	9
5.2.3	Tavily collector	10
5.2.4	Straightforward collection	10
5.2.5	Cleaning, deduplication and IDs	10
5.3	<code>pipeline.py</code> : orchestration layer	10
5.3.1	Simple function arguments	10
5.3.2	Query normalization	10
5.3.3	Shared state	10
5.3.4	Progress callback	11
5.4	<code>app.py</code> : presentation and interaction layer	11
5.4.1	Why session state?	11
5.4.2	Cloud secrets bridge	11

5.5	Supporting files	11
6	Tutorial Ke Four Stages: Role, Input, Output, Upgrade	12
6.1	1. Search Agent	12
6.2	2. Reader Agent	12
6.3	3. Writer Chain	12
6.4	4. Critic Chain	12
7	Complete Runtime Flow: Button Click Se Report Tak	13
7.1	Depth ka effect	13
8	Evidence, Citations and Hallucination Control	13
8.1	Is project RAG hai?	13
8.2	Stable citation IDs	14
8.3	What this guarantees – and what it does not	14
8.4	Prompt injection defense	14
9	Most Important Problems Faced and Fixes	14
9.1	Problem 1: Groq HTTP 413 / 8K token request	14
9.2	Problem 2: OpenAlex relevance weak thi	15
9.3	Problem 3: Python externally-managed environment	15
9.4	Problem 4: Streamlit Secrets invalid TOML	15
9.5	Problem 5: Deprecated provider model	15
9.6	Problem 6: Git remote and non-fast-forward push	15
10	Security and Responsible Engineering	15
10.1	Secrets management	15
10.2	Network safety and resilience	16
10.3	Input/output safety	16
10.4	Production gaps	16
11	Testing and Verification Strategy	16
11.1	What automated tests should be added next?	16
12	Deployment: Local Se Production Tak	17
12.1	Local	17
12.2	Streamlit Community Cloud	17
12.3	Docker	17
12.4	Scaling beyond single-process Streamlit	17
13	Design Decisions and Trade-offs	18
13.1	Complexity intuition	18

14 Interview Questions and Model Answers	18
14.1 Core architecture	18
14.2 Retrieval and evidence	19
14.3 LLMs and prompting	20
14.4 Python and backend	21
14.5 Frontend and deployment	21
14.6 Critical thinking	22
15 Two-Minute Demo Script	22
16 Resume Bullets and STAR Stories	23
16.1 Resume bullets	23
16.2 STAR story: product evolution	23
16.3 STAR story: reliability	23
17 Future Roadmap: Interview Mein Kya Bolna Hai	24
17.1 Phase 1: evidence quality	24
17.2 Phase 2: evaluation and observability	24
17.3 Phase 3: production scale	24
17.4 Phase 4: deeper agent workflow	24
18 Rapid Revision Cheat Sheet	24
18.1 Five lines you must be able to say	25
19 Glossary	25
20 Final Interview Checklist	25

1 Sabse Pehle: 60-Second Project Summary

One-line definition

InsightForge original tutorial ke simple **Search Agent** → **Reader Agent** → **Writer Chain** → **Critic Chain** template par bana multi-agent research platform hai. Yeh recent academic papers, news aur optional web sources collect karke cited report aur independent feedback generate karta hai.

1.1 Problem kya solve hoti hai?

Normal search engine links de deta hai, aur normal chatbot fluent answer de deta hai. Dono mein ek gap hai:

- researcher ko recent aur relevant evidence chahiye;
- student ko complex topic ka structured overview chahiye;
- founder ya policy professional ko trend ke saath counter-evidence bhi chahiye;
- har important claim ka source traceable hona chahiye;
- ek hi LLM ke bias aur hallucination ko blindly trust nahi karna chahiye.

InsightForge is gap ko evidence-first workflow aur specialized agents se solve karta hai.

1.2 English elevator pitch

“I started with a tutorial-based Search, Reader, Writer, and Critic workflow, then upgraded it with live academic and news sources, source IDs, a responsive UI, and deployment support. Gemini handles the first three stages and Groq provides an optional independent Critic.”

1.3 Hinglish elevator pitch

“Maine tutorial ke Search Agent, Reader Agent, Writer Chain aur Critic Chain ko base rakha. Search live papers/news laata hai, Reader evidence compare karta hai, Writer cited report banata hai aur Groq Critic weak claims check karta hai. Code ko jaan-boojhkar direct aur interview-friendly rakha hai.”

1.4 Project ke strongest differentiators

1. **Live evidence, not model memory only.**
2. **Independent review:** Groq available ho to completed report ko separately check karta hai.
3. **Simple model rule:** Gemini research, Groq critic; no routing matrix.
4. **Stable source IDs** report ko auditable banate hain.
5. **Graceful degradation:** source ya provider fail ho to poora app unnecessarily crash nahi karta.
6. **Production readiness:** responsive UI, secrets, Docker, health check, deployment guide.

2 Project Kahan Se Start Hua: Actual Evolution

2.1 Starting point

Original repository ek basic multi-agent research demo tha. Usmein broadly chaar ideas the:

- Search Agent web results leta tha.
- Reader Agent ek URL scrape karta tha.
- Writer Chain report banati thi.
- Critic Chain score aur feedback deti thi.

Yeh learning demo achha tha, lekin real product ke liye limitations thi: Tavily dependency, single scraped page, generic report, no strong evidence library, no source IDs, aur demo-like UI.

2.2 Idea pivots aur final requirement

Conversation mein pehle daily-life assistant aur education assistant jaise ideas explore hue. Final requirement clear hui:

“Aisa product jo researcher, student ya kisi bhi domain mein research karne wale insaan ko latest trends analyze karne mein help kare.”

Is requirement se product ke five pillars nikle:

1. Domain-agnostic research question.
2. Recent sources with configurable time window.
3. Trend analysis, not only summarization.
4. Counter-evidence and uncertainty.
5. Proper UI and deployability.

2.3 Actual refactoring order

Repository mein files pehle se thi, isliye unhe zero se random order mein create nahi kiya gaya; unko product architecture ke according refactor kiya gaya. Actual logical order yeh tha:

Step	File / concern	Kya banaya gaya
1	Product scope	Generic report generator ko universal research intelligence product define kiya.
2	<code>agents.py</code>	Original two-agent/two-chain contract preserve karke grounded prompts aur provider abstraction add ki.
3	<code>tools.py</code>	OpenAlex, Google News RSS aur optional Tavily collectors banaye.
4	<code>pipeline.py</code>	Original four sequential stages, simple dictionary state and progress callback banaye.
5	<code>app.py</code>	Responsive Streamlit experience, progress, tabs, evidence cards aur follow-up chat banaya.

6	Reliability	Query normalization, source timeouts, bounded snippets and deduplication add hue.
7	Models	Simple rule rakha: Gemini research aur optional Groq critic.
8	Deployment	Requirements, secrets examples, Streamlit config, Dockerfile, runtime and docs add hue.
9	Verification	Syntax, utility, live source, full pipeline, health endpoint aur AppTest checks hue.

Interview clarity

Agar interviewer pooche “pehli file kaunsi banayi?”, honest answer yeh hai: tutorial repository mein `agents.py`, `tools.py`, `pipeline.py` aur `app.py` skeleton already tha. Humne same learning order aur names preserve kiye: `tools/agents` samjhe, four-step pipeline improve ki, phir UI aur deployment add kiya.

3 Requirements Se Architecture Tak

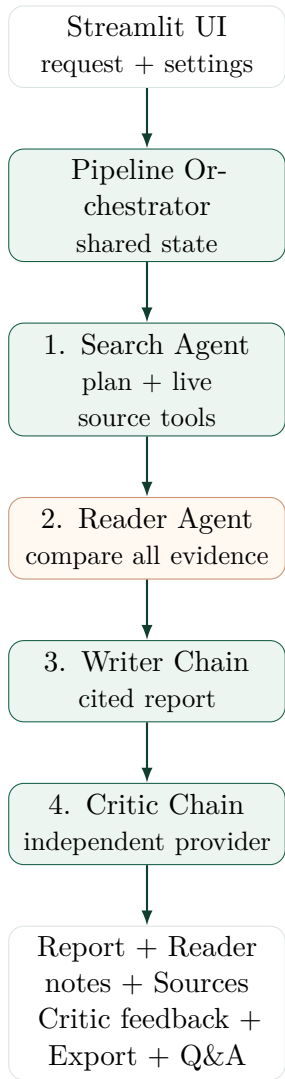
3.1 Functional requirements

- User research question, time range, depth, language, domain, region aur audience de sake.
- Academic papers and recent news default source hon.
- Tavily key ho to general web optional source ho.
- Output mein report, Reader notes, source library, Critic feedback and Search plan ho.
- User source-grounded follow-up question pooch sake.
- Full report Markdown mein download ho sake.
- English, Hinglish aur Hindi output support ho.

3.2 Non-functional requirements

- Responsive and simple UI.
- Network calls ke liye timeouts and isolated failures.
- API keys source control se bahar.
- Token/rate-limit aware prompts.
- Traceability through source IDs.
- Simple, explainable model selection.
- Local, Streamlit Cloud and Docker deployment.

3.3 High-level architecture



3.4 Why sequential pipeline?

Stages ka order tutorial jaisa dependency-driven hai. Reader ko Search results chahiye, Writer ko Reader notes chahiye, aur Critic ko completed report chahiye. Simplicity ke liye source calls bhi seedhe one-by-one run hote hain.

Interview line: “I kept the pipeline deliberately sequential so its execution order is obvious in an interview and easy to debug.”

4 Technology Stack: Kya Use Kiya Aur Kyun

Layer		Technology	Reason
Frontend		Streamlit	Python-only rapid product UI, forms, progress status, tabs, chat, download and cloud deployment.
Prompt tion	orchestra-	LangChain Core	<code>ChatPromptTemplate</code> , runnable pipe operator and <code>StrOutputParser</code> .

Primary synthesis	Google Gemini	Evidence-heavy context and strong synthesis for Standard/Deep runs.
Independent review	Groq-hosted model	Fast second-provider adversarial review; reduces single-model correlation.
Academic source	OpenAlex REST API	Recent scholarly metadata, abstracts, authors, venue, date, citation count; no required API key.
News source	Google News RSS	Recent reporting through RSS/XML without a dedicated paid key.
Optional web source	Tavily API	General web snippets for domains not fully covered by papers/news.
HTTP/XML	Requests + Element-Tree	Lightweight network calls and standard-library RSS parsing.
Configuration	python-dotenv st.secrets	+ Same application works locally and on Streamlit Cloud.
Packaging	requirements.txt	Reproducible Python dependencies.
Deployment	Streamlit Cloud Docker	+ Easy public hosting plus portable container option.
Version control	Git/GitHub	Source history, deployment integration and automatic redeploy on push.

4.1 Why not LangGraph?

Current workflow ek fixed DAG hai without looping, human approval, checkpointing or branch recovery. Simple functions and a state dictionary are easier to explain and maintain. LangGraph tab justified hoga jab iterative search, human-in-the-loop, persistent checkpoints, retries per node, ya conditional branching add karenge.

Interview line: “I avoided framework over-engineering. The workflow is deterministic enough for plain orchestration; LangGraph becomes valuable when the state machine becomes genuinely dynamic.”

5 File-by-File Deep Walkthrough

5.1 agents.py: intelligence layer

Is file ko original project ki tarah direct rakha gaya hai:

1. Environment variables load karna.
2. Gemini main model aur optional Groq critic banana.
3. Four prompt templates define karna.
4. Har prompt ko `prompt | model | StrOutputParser()` chain mein badalna.
5. Evidence-bound follow-up answer banana.

5.1.1 Simple model setup

Project mein koi provider routing matrix nahi hai. Gemini Search, Reader aur Writer chalata hai. Groq key ho to Groq Critic chalata hai; key na ho to Gemini Critic bhi chala deta hai. Yeh ek hi simple fallback hai.

Listing 1: Actual model selection idea

```

1 main_model = ChatGoogleGenerativeAI(...)
2
3 if GROQ_API_KEY:
4     critic_model = ChatGroq(...)
5 else:
6     critic_model = main_model

```

Why optional second provider? Independent Critic useful hai, but application ko samajhne ke liye complex router zaroori nahi. Interview answer seedha hai: “Gemini research karta hai; Groq report review karta hai.”

5.1.2 Prompt design principles

- Role clearly specified.
- Input sections named: request, search plan, evidence, reader notes and report.
- Output structure fixed using Markdown headings.
- Evidence boundary explicit: only [S3] style supplied IDs cite karo.
- Uncertainty aur insufficient evidence clearly state karo.
- Temperature 0.15: creative prose se zyada stable analytical output.

5.2 tools.py: live evidence layer

Yeh file model-independent data acquisition karti hai.

Video se direct mapping ke liye `web\_search(query)` aur `scrape\_url(url)` names ab bhi available hain. Deployed pipeline structured cards ke liye richer `collect\_evidence()` use karti hai. Iska matlab concept replace nahi hua; original tool ko production data contract mila.

5.2.1 OpenAlex collector

- Current date minus selected days se `from_publication_date` filter banta hai.
- Date window recency guarantee karti hai; window ke andar results relevance score se sort hote hain.
- Title, DOI/landing URL, publication date, venue, authors and cited-by count normalize hote hain.
- OpenAlex abstract inverted index deta hai, jise position-word pairs sort karke normal abstract mein rebuild kiya jata hai.

Listing 2: Inverted abstract reconstruction concept

```

1 words = []
2 for word, positions in inverted.items():
3     words.extend((position, word) for position in positions)
4 abstract = " ".join(word for _, word in sorted(words))

```

5.2.2 Google News RSS collector

Query mein `when:Nd` add hota hai. XML response ko `ElementTree` parse karta hai. Title, link, date, publisher and description standardized record mein map hote hain. RSS free and convenient hai, lekin link Google redirect ho sakta hai aur snippet full article nahi hota.

5.2.3 Tavily collector

TAVILY_API_KEY present ho tabhi job schedule hoti hai. Yeh general web coverage deta hai. Key absent ho to academic/news pipeline normally chalti rehti hai.

5.2.4 Straightforward collection

Search functions ek simple loop mein run hote hain. Har selected source ka function call visible hai, isliye beginner easily debug kar sakta hai.

- Ek source fail ho to baaki evidence retain hota hai.
- Error short warning mein convert hota hai.
- Trade-off: parallel version se thodi zyada latency ho sakti hai, lekin code ka learning cost kam hai.

5.2.5 Cleaning, deduplication and IDs

HTML tags and repeated whitespace remove hote hain. Summary 500 characters tak bounded hai. URL otherwise title identity se duplicates remove hote hain. Final records ko deterministic run-local IDs S1, S2, ... milte hain.

5.3 pipeline.py: orchestration layer

5.3.1 Simple function arguments

Dataclass aur request adapters remove kar diye gaye. Main function seedhe topic aur optional form values leta hai:

```
run\_research\_pipeline(topic)
```

Time range, domain, language aur source toggles normal keyword arguments hain. Yeh original video ke call ke sabse close hai.

5.3.2 Query normalization

Natural question OpenAlex ko over-constrain kar sakta tha. `_build_search_query()` punctuation and common question/recency stopwords remove karta hai, duplicate terms hataata hai, important short token `ai` preserve karta hai, aur query ko first 12 terms tak limit karta hai.

Example:

```
What are recent advances in small language models for on-device AI?
↓
small language models device ai
```

5.3.3 Shared state

Pipeline ek dictionary mein inspectable artifacts rakhti hai:

```
1 state = {
2     "request": {...},
3     "models": {"research": "Gemini", "critic": "Groq"},
4     "search_plan": "...",
```

```

5     "search_results": "...",
6     "evidence": [...],
7     "scraped_content": "...",
8     "report": "...",
9     "feedback": "..."
10 }

```

This is useful for UI tabs, debugging, export and future persistence.

5.3.4 Progress callback

Pipeline Streamlit import nahi karti. Instead optional callback ko **step**, **status** deti hai. Is dependency inversion se core pipeline CLI/test mein bhi run hoti hai, aur UI apna progress visualization decide karti hai.

5.4 app.py: presentation and interaction layer

Responsibilities:

- Page config and responsive CSS.
- Local env plus Streamlit Cloud secrets bridge.
- Example questions and compact research form.
- Advanced domain/region/audience/source settings.
- Four-stage progress bar without duplicate log rows.
- Report, Reader notes, Sources, Critic feedback and Search plan tabs.
- Evidence cards with safe HTML escaping and validated link schemes.
- Report download and evidence-bound follow-up chat.
- Session state for result and conversation continuity.

5.4.1 Why session state?

Streamlit every interaction par script top-to-bottom rerun karta hai. Without `st.session_state`, generated report aur follow-up history lost ho jati. Result and messages session state mein store hone se reruns ke across survive karte hain.

5.4.2 Cloud secrets bridge

Local code `os.getenv()` use karta hai, jabki Streamlit Cloud credentials `st.secrets` mein deta hai. App startup selected secret names ko environment mein `setdefault` karta hai. Local environment priority retain hoti hai; cloud par same backend code reuse hota hai.

5.5 Supporting files

File	Purpose
<code>requirements.txt</code>	Streamlit, LangChain provider integrations, dotenv and requests with version ranges.

<code>.env.example</code>	Safe configuration template; real keys absent.
<code>.gitignore</code>	Real <code>.env</code> , <code>virtualenv</code> , <code>cache</code> and local <code>Streamlit</code> secrets exclude karta hai.
<code>.streamlit/config.toml</code>	Light theme, server security flags and usage telemetry preference.
<code>.streamlit/secrets.toml.example</code>	Cloud/local TOML secret format example.
<code>Dockerfile</code>	Python 3.12 slim image, dependency layer, port 8501, health check, <code>Streamlit</code> command.
<code>.dockerignore</code>	Secrets, Git metadata, <code>venv</code> and <code>cache</code> image context se exclude karta hai.
<code>runtime.txt</code>	Cloud Python version intent.
<code>README.md</code>	Product summary, workflow and local run instructions.
<code>DEPLOYMENT.md</code>	<code>Streamlit</code> Cloud and <code>Docker</code> deployment steps.

6 Tutorial Ke Four Stages: Role, Input, Output, Upgrade

6.1 1. Search Agent

Input: complete research profile.

Output: search objective, sub-questions, evidence criteria, blind spots, normalized query and live records.

Tutorial connection: original Search Agent Tavily tool call karta tha. Upgraded stage same responsibility rakhti hai, lekin `OpenAlex` + `Google News` + optional Tavily use karti hai aur structured [Sx] IDs deti hai.

6.2 2. Reader Agent

Input: Search plan + all retrieved records.

Output: established findings, emerging signals, disagreements, limitations and evidence gaps.

Tutorial connection: original Reader ek selected URL scrape karta tha. Production version saare bounded abstracts/snippets compare karta hai, kyunki one-page selection brittle aur biased ho sakta hai. Original `scrape\_url()` helper learning compatibility ke liye available hai.

6.3 3. Writer Chain

Input: request profile + Reader notes + exact source records.

Output: audience-specific cited report with key findings, trends, uncertainty, implications and method note.

Tutorial connection: same `LangChain` composition—`prompt` | `llm` | `StrOutputParser()`—use hoti hai; prompt ko evidence IDs aur uncertainty rules se strengthen kiya gaya.

6.4 4. Critic Chain

Input: completed report + exact source records.

Output: score, strengths, claims to improve, missing/conflicting evidence and one-line verdict.

Checks: unsupported citations, selection bias, duplicate reporting, correlation vs causation, date/geography mismatch and overconfidence. Dono keys ho to Critic independent provider use karta hai.

Important honest limitation

Tutorial order preserve karne ke liye Critic completed report ke baad feedback deta hai. Current report automatically rewrite nahi hoti. Future optional revision loop Critic feedback ko Writer mein wapas bhej sakta hai, lekin tab workflow tutorial ke simple four-stage path se aage badhega.

7 Complete Runtime Flow: Button Click Se Report Tak

1. User form fill karke **Start research** click karta hai.
2. App empty question and empty source selection validate karti hai.
3. UI values direct pipeline function arguments bante hain.
4. Pipeline Gemini main model aur optional Groq critic banati hai.
5. Search Agent question ko scope karta hai aur plan banata hai.
6. Natural question keyword query mein normalize hoti hai.
7. Search stage selected source functions one-by-one call karti hai.
8. Records clean, deduplicate and source-ID assign hote hain.
9. Reader Agent saare bounded records compare karke research notes banata hai.
10. Writer Chain notes + evidence se cited report banati hai.
11. Critic Chain, preferably another provider, completed report stress-test karti hai.
12. UI state ko metrics and five result tabs mein render karti hai.
13. Export full report + source library + audit trail combine karta hai.
14. Follow-up chat same evidence boundary ke andar answer karti hai.

7.1 Depth ka effect

Depth	Per selected source limit	Research model
Quick scan	4	Gemini
Standard	6	Gemini
Deep dive	8	Gemini

If academic + news selected hain, Standard roughly up to 12 raw records la sakta hai before deduplication.

8 Evidence, Citations and Hallucination Control

8.1 Is project RAG hai?

Strict textbook vector-RAG nahi hai because embeddings, chunk index aur vector database use nahi ho rahe. Yeh **live retrieval-augmented research pipeline** hai: external evidence retrieve hota hai, prompt context mein inject hota hai, aur grounded synthesis hoti hai.

Interview line: “It is retrieval-augmented generation, but not vector-store RAG. Retrieval is live and API-based rather than embedding-based.”

8.2 Stable citation IDs

Har run mein evidence record ko S1, S2 ID milta hai. Prompts model ko only supplied IDs cite karne bolte hain. UI evidence library mein same ID show hoti hai. Isse user claim se source card tak trace kar sakta hai.

8.3 What this guarantees – and what it does not

Improves:

- traceability;
- invented URLs reduce karna;
- missing evidence explicitly show karna;
- claims ko review karna.

Does not guarantee:

- source itself correct hai;
- model ne source ko perfectly interpret kiya;
- every citation semantically entails the full sentence;
- search coverage exhaustive hai.

Future production version mein citation entailment checker, source quality scoring and full-text verification add karna chahiye.

8.4 Prompt injection defense

Retrieved snippets untrusted hain. System prompt explicitly bolta hai: evidence ke andar commands ko ignore karo, unhe instructions nahi data treat karo. UI external text ko HTML render karne se pehle `html.escape()` karti hai. Link only `http://` or `https://` scheme par button banta hai.

9 Most Important Problems Faced and Fixes

9.1 Problem 1: Groq HTTP 413 / 8K token request

Symptom: review stage ne approximately 8319 tokens request kiye, account limit 8000 thi.

Root cause: report + evidence + prompts ka combined context too large tha.

Fixes:

- Groq default large model se lower-throughput-friendly model par shift.
- Output tokens 1400 cap.
- Per-source excerpt 500 chars.
- Quick/Standard/Deep source limits reduce.
- Direct string slices before Reader, Writer and Critic calls.

Verification: Standard pipeline live sources aur all four tutorial-aligned stages ke saath successfully run hui.

STAR version: Situation – production-like run Critic stage par 413 se fail hua. Task – request ko provider limit ke andar lana. Action – source count, excerpt size, prompt input and output size directly cap ki. Result – same four-stage flow smaller predictable context ke saath run hui.

9.2 Problem 2: OpenAlex relevance weak thi

Symptom: long natural-language question par zero ya tangential papers.

Fix: keyword normalization, date filter + relevance sorting.

Lesson: user query and retrieval query same cheez nahi hote.

9.3 Problem 3: Python externally-managed environment

System pip ne PEP 668 error diya. Fix project-local virtual environment:

```
1 python3 -m venv .venv
2 .venv/bin/python -m pip install -r requirements.txt
3 .venv/bin/python -m streamlit run app.py
```

9.4 Problem 4: Streamlit Secrets invalid TOML

Model name slash/hyphen string without quotes invalid TOML tha.

```
1 # Wrong
2 GROQ_MODEL=openai/gpt-oss-20b
3
4 # Correct
5 GROQ_MODEL = "openai/gpt-oss-20b"
```

9.5 Problem 5: Deprecated provider model

Hardcoded model IDs time ke saath retire ho sakte hain. Isliye model names environment variables mein configurable rakhe gaye. Interview mein exact model se zyada important point hai ki model name source code edit kiye bina change ho sakta hai.

9.6 Problem 6: Git remote and non-fast-forward push

Original clone another repository ko point karta tha. Remote user repository par switch hua. Later GitHub ne Dev Container commit add kiya, isliye push reject hua. Safe response: remote fetch, inspect, rebase, then normal push. Force push avoid hua.

10 Security and Responsible Engineering

10.1 Secrets management

- Real `.env` Git ignored hai.
- Cloud secrets hosting dashboard mein encrypted config ke through set hote hain.
- `.env.example` only placeholders rakhta hai.
- Docker build context real env and secrets exclude karta hai.

- Exposed API key ko immediately revoke/rotate karna chahiye.

10.2 Network safety and resilience

- OpenAlex/news timeout 18 seconds; Tavily 22 seconds.
- `raise_for_status()` HTTP failure silently ignore nahi karta.
- Expected errors per-source warnings bante hain.
- User-Agent identify karta hai ki call local research assistant se hai.

10.3 Input/output safety

- External titles/snippets HTML-escaped.
- Source links scheme-checked.
- Prompt evidence treated as untrusted.
- High-stakes findings primary sources se verify karne ka disclaimer.
- No claim that model output is guaranteed truth.

10.4 Production gaps

SSRF risk low hai because current collectors fixed endpoints use karte hain, but future arbitrary URL scraping add ho to private IP ranges, redirects, content type and response size validate karne honge. Public multi-user app ke liye authentication, quotas, abuse controls and per-user isolation add karni hogi.

11 Testing and Verification Strategy

Project development mein following checks use hue:

Test	Purpose
<code>python -m py_compile</code>	Syntax errors across app, agents, pipeline and tools.
Utility assertions	Cleaning, query normalization, evidence formatting and export IDs.
Provider construction	Correct model and max token configuration without full run.
Live OpenAlex/news test	External collectors returned structured source records.
Single agent call	Provider SDK + prompt chain integration.
Full Standard pipeline	All agent stages and export with live evidence.
Streamlit health endpoint	Server process bound and served health route.
Streamlit AppTest	App rendered with zero exceptions; widgets detectable.
<code>git diff --check</code>	Whitespace/patch hygiene.

11.1 What automated tests should be added next?

1. Mock requests for OpenAlex, RSS and Tavily response parsing.
2. Query normalizer parameterized tests.

3. Gemini/Groq model creation tests.
4. Pipeline tests with mocked agents and collector failures.
5. Citation validator: every [Sx] in report exists in evidence.
6. Prompt-injection regression cases.
7. Snapshot tests for exported Markdown sections.
8. LLM eval dataset for correctness, coverage, calibration and citation support.

12 Deployment: Local Se Production Tak

12.1 Local

```
1 git clone https://github.com/Kaushik9560/Research-Mind.git
2 cd Research-Mind
3 python3 -m venv .venv
4 .venv/bin/python -m pip install -r requirements.txt
5 .venv/bin/python -m streamlit run app.py
```

12.2 Streamlit Community Cloud

1. GitHub repo and main branch choose karo.
2. Entrypoint app.py set karo.
3. Python 3.12 select karo.
4. Secrets TOML mein rotated keys and model names add karo.
5. Deploy; later Git push automatic redeploy trigger karega.

```
1 GROQ_API_KEY = "rotated-key"
2 GROQ_MODEL = "openai/gpt-oss-20b"
3 GOOGLE_API_KEY = "rotated-key"
4 GOOGLE_MODEL = "gemini-3.6-flash"
5 MAX_OUTPUT_TOKENS = 1400
```

12.3 Docker

Dockerfile dependency installation ko source copy se pehle karta hai, so dependency layer cache ho sakti hai. Container port 8501 expose karta hai aur `/_stcore/health` health check use karta hai.

```
1 docker build -t insightforge .
2 docker run --rm -p 8501:8501 --env-file .env insightforge
```

12.4 Scaling beyond single-process Streamlit

Current pipeline synchronous hai. Long research ya many users ke liye:

- FastAPI API layer;
- Celery/RQ or managed queue;

- Redis job status/cache;
- PostgreSQL research sessions;
- object storage for exports;
- background workers and WebSocket/SSE progress;
- rate limiting and auth.

13 Design Decisions and Trade-offs

Decision	Benefit	Trade-off
Live APIs instead of vector DB	Fresh trends, no indexing pipeline	Coverage depends on upstream search and snippets.
Sequential agents	Clear dependencies and audit trail	Multiple LLM calls increase latency/cost.
Optional Groq review	More independent critique when configured	Second key is optional.
RSS for news	No extra paid API key	Redirect links, snippet-only content, locale bias.
Streamlit	Very fast product UI	Less control than React/FastAPI architecture.
Sequential source calls	Easy to read and debug	Slower than parallel I/O.
Markdown output	Easy render/export	Structured downstream analytics harder than strict JSON.
Direct string limits	Simple context-size guard	Character count is not exact token count.
Run-local IDs	Simple traceability	IDs do not persist across repeated runs.
Low temperature	Stable analytical answers	Less diversity in phrasing/exploration.

13.1 Complexity intuition

Let S be number of selected source collectors and N total returned records.

- Source wall time is approximately $O(T_1 + \dots + T_S)$ because calls are sequential.
- Deduplication is expected $O(N)$ using a set.
- Abstract reconstruction is $O(W \log W)$ due to sorting word positions.
- LLM latency dominates end-to-end runtime because four model stages are sequential.

14 Interview Questions and Model Answers

14.1 Core architecture

Q1. Why did you use multiple agents instead of one large prompt?

I preserved the tutorial's separation of concerns: Search plans and retrieves, Reader compares evidence, Writer communicates, and Critic audits. Intermediate artifacts are inspectable and testable. The trade-off is added latency and token cost compared with one prompt.

Q2. Are these autonomous agents?

They are specialized LLM-powered stages in a controlled orchestration, not fully autonomous agents with unrestricted planning. I deliberately kept the workflow bounded for reliability and auditability.

Q3. Why are live source calls deterministic even though it is called Search Agent?

The Search stage combines an LLM search plan with deterministic HTTP/API tools. Retrieval itself should be observable and reproducible; the model handles semantic scoping, not fabrication of search results.

Q4. How does data move between agents?

A normal dictionary stores request data and each stage's output. Every stage reads the previous key and adds its own key.

Q5. Why not run all agents in parallel?

Each stage needs the previous result. I also kept source calls sequential because this is a learning-focused project; the small latency trade-off makes the code much easier to explain and debug.

14.2 Retrieval and evidence

Q6. Why OpenAlex?

It provides broad scholarly metadata, search, dates, authors, venues, DOI links, cited-by counts and many abstracts through an accessible API. It is a strong zero-key academic baseline.

Q7. Does citation count prove paper quality?

No. It is age- and field-dependent and can be gamed. The prompt explicitly prevents using citation count alone as quality proof.

Q8. How do you identify latest trends?

First constrain a time window, then examine repeated signals across independent sources, research activity, adoption, policy/market drivers and counter-signals. Recency alone is not called a trend.

Q9. How do you ensure citations are real?

URLs and IDs originate from collector records, and prompts may cite only supplied IDs. A remaining improvement is automated entailment validation for every claim-citation pair.

Q10. Why not scrape full articles?

Full-text scraping creates robots, paywall, copyright, latency, parsing and security issues. Metadata/snippets are a safe baseline. Future full-text access should prefer official APIs and licensed/open-access content.

Q11. What happens if one source fails?

Each source call has a small try/except. A failed API becomes a warning and the next selected source still runs.

14.3 LLMs and prompting**Q12. Why temperature 0.15?**

Research synthesis benefits from consistency and lower variation. It is not zero because some flexible organization is useful, but factual grounding matters more than creativity.

Q13. Why Gemini plus Groq?

Gemini handles Search, Reader and Writer. Groq handles the Critic so the report can be reviewed by a different model. This is a fixed, easy-to-explain rule rather than automatic routing.

Q14. What if one API key is absent?

Google key is required because Gemini runs the research stages. Groq is optional; without it Gemini also runs the Critic. There are no additional provider fallbacks.

Q15. How do you control hallucination?

Live evidence, stable source IDs, explicit evidence rules, low temperature, uncertainty instructions, and an independent Critic that audits the completed report. I still communicate that this reduces but cannot eliminate hallucination.

Q16. What is prompt injection here?

A retrieved snippet might contain text instructing the model to ignore system rules. The agent prompt labels evidence as untrusted data and tells the model to ignore instructions inside it. Production hardening can add content classifiers and stricter structured extraction.

Q17. Why use LangChain?

It gives a consistent provider interface and composable prompt-model-parser pipeline. Business orchestration remains ordinary Python, so lock-in is limited.

Q18. How did you manage context limits?

Bounded record counts, 500-character excerpts, direct string slices before each model call, and a maximum output-token setting.

14.4 Python and backend

Q19. Why normal function arguments instead of a request class?

The project is small and interview-focused. Explicit function arguments keep the main pipeline call close to the tutorial and avoid introducing a class that adds little value here.

Q20. Why are source calls sequential?

The application calls only two or three sources. Sequential requests make control flow obvious. If latency becomes important, parallel I/O is a measured future optimization.

Q21. Why did you remove automatic provider routing and client caching?

They made a small learning project harder to explain. A fixed Gemini-research/Groq-critic rule is enough. I would add caching or routing only after profiling a real production workload.

Q22. Why expected exceptions only in collectors?

Network and parse failures are recoverable. Catching every exception would hide programming defects such as KeyError or schema bugs, making production debugging harder.

Q23. How does deduplication work?

It uses normalized URL, falling back to title, as identity in a set. This is $O(N)$ expected time, but semantic duplicate stories with different URLs remain a future improvement.

14.5 Frontend and deployment

Q24. Why Streamlit?

It let me ship a Python-native research product quickly with forms, status, tabs, chat and download. If product complexity grows, I would separate FastAPI backend and a React frontend.

Q25. How does state survive Streamlit reruns?

Generated output and follow-up messages live in `st.session_state`. Without it every widget interaction would reconstruct an empty screen.

Q26. How are secrets different locally and in cloud?

Local development loads `.env`; Streamlit Cloud provides `st.secrets`. Startup bridges approved secret names into environment variables, so backend provider code stays unchanged.

Q27. Why Docker health check?

It lets an orchestrator distinguish a running process from a ready Streamlit service and restart unhealthy containers.

Q28. What caused the externally managed environment error?

PEP 668 prevents pip from mutating the distribution-managed Python environment. A project virtual environment is the safe solution; bypass flags can break system packages.

14.6 Critical thinking

Q29. What is the biggest current limitation?

Evidence coverage and verification depth. The system mostly uses metadata/snippets, and model citations are not automatically checked for entailment. That is the first area I would strengthen.

Q30. How would you measure quality?

Build a benchmark of research questions with expert-labeled claims and sources. Track source precision/recall, citation validity, claim support, contradiction coverage, calibration, latency and cost.

Q31. How would you reduce cost?

Cache retrieval, hash and reuse identical evidence, route quick scans to cheaper models, reduce repeated context, summarize evidence once, batch evals, and enforce per-user quotas.

Q32. How would you improve search?

Generate multiple structured subqueries from the Search Agent plan, search each, rerank with a cross-encoder, diversify by source/domain/date, and stop when evidence saturation is reached.

Q33. Is an LLM critic actually reliable?

It is a heuristic reviewer, not a proof system. Cross-provider critique improves diversity, but deterministic citation checks and human review remain necessary for high-stakes research.

Q34. What happens under many simultaneous users?

Provider quotas and synchronous worker capacity become bottlenecks. I would move jobs to a queue, persist state, stream progress, add caching, rate limits and per-user quotas.

Q35. Why is this a real-world project rather than a demo?

It integrates live sources, evidence traceability, failure handling, token budgeting, independent review, secure configuration, responsive UI, exports, deployment and explicit limitations—not only a single prompt call.

15 Two-Minute Demo Script

Interview demo mein random clicking avoid karo. Yeh sequence follow karo:

1. **Problem (15 sec):** “Search gives links, chatbot gives unverified prose; this product joins live evidence with structured critique.”
2. **Input (15 sec):** Example select karo; 1 year, Standard, English. More options mein sources show

karo.

3. **Stages (20 sec):** Four tutorial-aligned cards explain karo. Start research press karo and progress bar point out karo.
4. **Brief (25 sec):** Executive answer and inline [Sx] citations show karo.
5. **Evidence (20 sec):** Same IDs, publication dates and original source links show karo.
6. **Critical review (15 sec):** Weak claims and missing evidence show karo.
7. **Models (15 sec):** simple rule explain karo—Gemini research, Groq review.
8. **Close (10 sec):** Export and follow-up question demonstrate karo.

Demo safety

Live APIs unpredictable hote hain. Interview se pehle ek successful result ready rakho. Agar network fail ho, architecture and graceful warning demonstrate karo instead of hiding the limitation.

16 Resume Bullets and STAR Stories

16.1 Resume bullets

- Built and deployed an evidence-first multi-agent research platform that synthesizes recent scholarly and news sources into citation-mapped trend reports.
- Upgraded a tutorial-based four-stage Search–Reader–Writer–Critic workflow with live evidence, source IDs, and optional cross-provider review.
- Used Gemini for research and an optional Groq critic, keeping model selection explicit and easy to maintain.
- Improved source reliability with timeouts, simple per-source error handling, deduplication, and stable source IDs.
- Shipped a responsive Streamlit UI with evidence library, progress tracking, follow-up Q&A, Markdown export, Docker health checks, and cloud secrets support.

16.2 STAR story: product evolution

Situation: Existing project was a generic search/scrape/write/critic demo.

Task: Convert it into a useful, domain-agnostic research product.

Action: Reframed requirements around evidence, trends, counter-signals, traceability and deployment; refactored tools, agents, orchestration and UI.

Result: A deployable system for researchers, students and decision makers with live sources and auditable outputs.

16.3 STAR story: reliability

Situation: Standard run failed on Groq with request-too-large error.

Task: Preserve useful evidence while meeting account limits.

Action: Measured stage context, bounded excerpts, capped outputs, used direct per-stage slices, and reduced source limits.

Result: Full Standard workflow passed with 12 live sources.

17 Future Roadmap: Interview Mein Kya Bolna Hai

17.1 Phase 1: evidence quality

- Multiple search queries per sub-question.
- Cross-encoder reranking and source diversity.
- DOI canonicalization and semantic duplicate clustering.
- Open-access full text through official APIs.
- Claim-citation entailment verifier.

17.2 Phase 2: evaluation and observability

- LangSmith/OpenTelemetry traces.
- Golden dataset and automated LLM evaluations.
- Cost, tokens, latency and source coverage dashboard.
- User feedback on useful/unsupported claims.

17.3 Phase 3: production scale

- Auth, teams and saved workspaces.
- Queue-backed asynchronous runs.
- PostgreSQL session/report persistence.
- Redis retrieval and answer cache.
- PDF/DOCX citation export and bibliography formats.
- Scheduled trend monitoring and change detection.

17.4 Phase 4: deeper agent workflow

Use LangGraph when query decomposition, iterative retrieval, evidence sufficiency decisions, human approval and checkpoint resume become necessary.

18 Rapid Revision Cheat Sheet

Remember this flow

Question → Search plan + live sources → Reader notes → Writer report → Critic feedback → Evidence-bound Q&A.

Keyword	One-line memory
Multi-agent	Specialized bounded stages, not unrestricted autonomy.
RAG	Live API retrieval augmentation, not vector-store RAG.
Grounding	Model receives actual source records and may cite only their IDs.

Adversarial review	Optional Groq Critic checks the completed report.
Execution	Stages and source calls are sequential for readability.
Token budget	Bounded source count + excerpts + direct slices + output cap.
Fallback	Only Groq is optional; Gemini is required and can also run the Critic.
State	Request and every intermediate artifact kept in an inspectable dictionary.
Security	Ignore instructions in evidence, escape HTML, validate URL schemes, hide secrets.
Deployment	Streamlit Cloud secrets or Docker environment variables.
Main limitation	Snippet-level evidence and no deterministic citation entailment yet.

18.1 Five lines you must be able to say

1. "I separate deterministic retrieval from probabilistic reasoning."
2. "The Critic independently audits the completed report; an optional revision loop is the next improvement."
3. "Citations are traceable IDs, but I do not claim that traceability alone guarantees truth."
4. "I kept the flow sequential so every stage is easy to explain and debug."
5. "The next production improvement is automated claim-citation verification and persistent async jobs."

19 Glossary

Term	Meaning
Agent	A model/tool stage with a specific role, inputs and output contract.
Orchestrator	Code that controls order, state transfer, progress and failures.
Evidence grounding	Generating analysis using retrieved records rather than memory only.
Counter-signal	Evidence that weakens, limits or contradicts a proposed trend.
Calibration	Matching confidence to actual evidence strength.
Prompt injection	Malicious instructions hidden inside retrieved content.
Reranking	Reordering retrieved results with a stronger relevance model.
Entailment	Whether a source actually supports a claim.
Rate limit	Provider restriction on requests/tokens over time.
Context window	Maximum input plus output token capacity of a model request.
Graceful degradation	Partial functionality continues when an optional component fails.
Idempotency	Repeating an operation produces no unintended duplicate effect.
Observability	Traces, metrics and logs that explain system behavior.

20 Final Interview Checklist

Interview se pehle confirm karo:

- ☐ 60-second English and Hinglish pitch yaad hai.
- ☐ Architecture without screen draw kar sakte ho.
- ☐ Har file ki one-line responsibility bol sakte ho.
- ☐ Why four stages, why Gemini + optional Groq, and why sequential flow explain kar sakte ho.
- ☐ 413 token-limit STAR story ready hai.
- ☐ Honest limitations and next improvements ready hain.
- ☐ One successful demo query and result ready hai.
- ☐ No key screen/share/history mein visible hai; leaked keys rotated hain.
- ☐ GitHub and deployed Streamlit links open ho rahe hain.
- ☐ Resume bullets metrics ke saath personalize kiye hain.

Best interview mindset: architecture ko defend karo, limitations ko hide mat karo, aur har design decision ka trade-off batao.